Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Numpy

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Numpy
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Numpy

1 Numpy

Study focus: Numpy. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000102 · EV0000103

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Today we are going to explore NumPy, one of the most powerful libraries in Python, especially when it comes to scientific and numerical computing.

Key details

- Imagine you're standing in front of a huge mountain of numbers.
- Numbers from experiments, data, from marks of students or images filled with pixels.
- And Python looks at you and says, I can help, but not alone.
- This is exactly where NumPy enters a story.
- NumPy is like giving Python a superpower, the power to handle large numerical data elegantly, efficiently, and unbelievingly fast.
- NumPy, short for numerical Python, is not just a library, it's the foundation on which most data science and machine learning libraries are built.
- Its heart is the NB array.
- Think of it as a disciplined, highly optimized container that holds data in a clean and organized manner.
- Like a neat cupboard where every item has the same size, same type, and it is perfectly arranged.
- Let me tell you a short story.
- Imagine having a list of a million numbers and you want to multiply every value by two.
- If Python lists try to do this, they get tired.
- They walk slowly.
- They use a lot of memory on the way.
- But Numbai, Numbai does it in a single breath.
- Because the work is done deep inside optimized C code.

- Think of arrays as building blocks.
- You can have a 1D array like a simple list of marks.
- We usually call it as vectors.
- A 2D array like a table, we represent it as mattress, rows and columns, and then we have an N-dimensional.
- Array we usually represent call it as tensors, like stacks of images or lay of data.
- Number arrays keep everything clean, fast and mathematical.
- Creating arrays feels almost like magic spells.
- You can see the functions here, np.z, np.1s, and np.a range.
- NP dot zeros create a clean slate of mattress of zeros.
- Likewise, np dot one gives you a table full of ones useful for testing.
- NP.a range gives you numbers from zero to nine with a wave of your hand.
- With just few lines, massive data sets can be manageable.
- Imagine having two lists and wanting to add them.
- In pure Python, you would be probably using a loop.
- But Numai says why look?
- When I can do it instantly.
- And you have A multiplied by two and you get the multiplication of the A array.
- Number performs lightning fast operations on entire arrays.
- If you need to sort your data, NumPy gently rearranges it with the help of the function called sort.
- And you need to pass the array.
- If you need to join two arrays,.

Example or application from the lecture

- Today let's take a small and meaningful journey into this magical library called NumPy.
- Let's talk about the basics of arrays.
- Let's now see how to create arrays.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000102 · EV0000103 · EV0000104 · +1 source units in pack.json

Core explanation · 05:00–09:59

You can use concatenate function.

Key details

- It connects them smoothly.
- talking about reshaping arrays, reshaping feels like transforming clay into new shapes without changing the amount of the clay.
- Take a 1D array twist it and suddenly it becomes a two by three matrix using reshape.
- Reshaping is heavily used in images, ML models and simulations.
- It's not just technical, it's elegant.
- Then talking about the array attributes, every array quietly carries important information about itself.

- Size tells you the total elements, detail tells you what data lives inside.
- It's like introducing the array itself politely before you start working with it.
- Next comes indexing and slicing.
- Indexing in NumPy feels like reading exactly the part of the story you want.
- You want the fourth number, then you can give array of three because array indexes starts from zero.
- You want numbers from two to four, then you can use slicing.
- In a 2D matrix, matrix of one, two is like saying bring me the element in the second row and call third column.
- row 0, row 1 and row 2, column 0, column 1 and column 2, and we are getting this one.
- Array indexes starts from 0.
- So this is the first row and the second column.
- And add boolean indexing, that's your filter.
- It quietly handles you only the values that matter.
- What are aggregate functions?
- Think of aggregation functions as small detectives.
- They scan your data and report back.
- Max gives you the largest value.
- Min gives you the smallest value in the array.
- The average of the array.
- S T D talks about the standard deviation, the average, sorry, the spread or variation in the array.
- This gives you instant insights with almost no effort.
- Python lists and python arrays.
- Python lists are regular school bags, flexible.
- You can put many things but they are get heavy.
- But NumPy RAs are like professionally designed travel bag, organized, lightweight, optimised and ready for long journeys.
- When you deal with serious numerical data, you pick the travel bag.
- It teaches y us to think in arrays, to avoid loops, to embrace speed and to enjoy clean mathematical expression.

Example or application from the lecture

- Let's end with a simple comparison story.
- Here is given the difference between the lists and arrays.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000106 · EV0000107 · EV0000108 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Numpy transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9995; only the JSON segmentation supplies the displayed timestamps.
Numpy transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/014_Numpy__7d37477e1600226d.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.

4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00038	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...
SRC00039	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00040	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...
SRC00041	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0037. Sidecar variants compare...

C Revision and retrieval practice